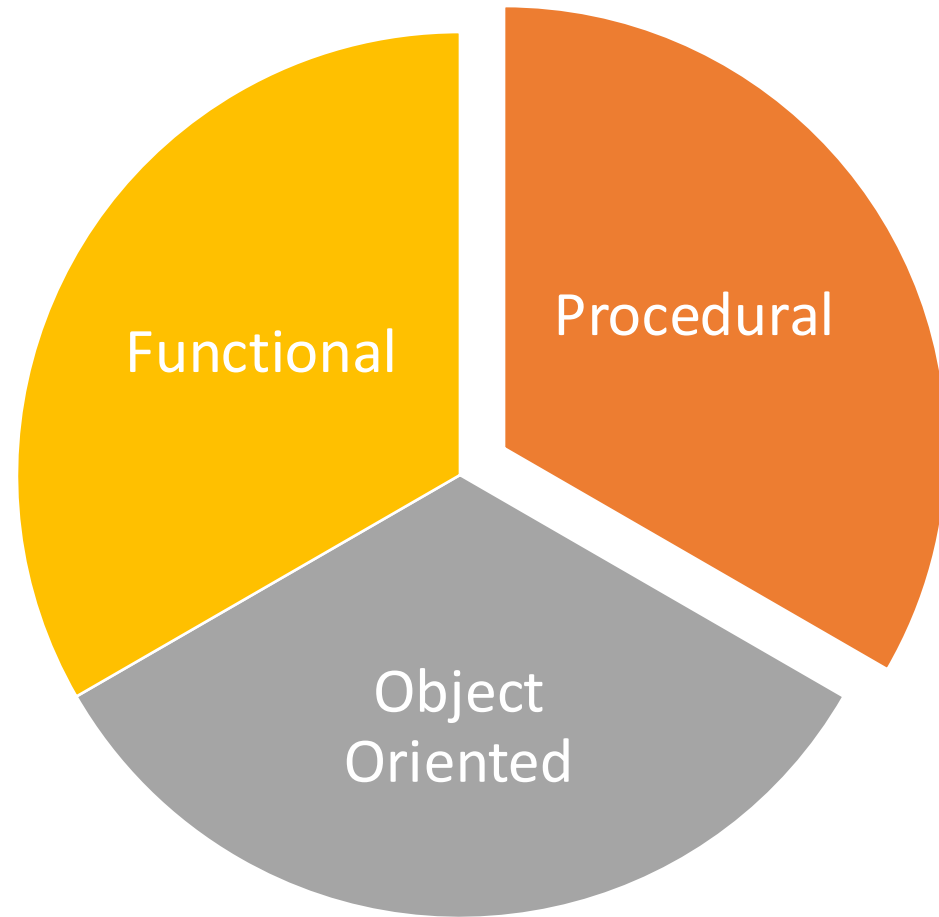


Go programming paradigm





Go vs OO

Object orientation and big software

Go is object-oriented.

Go does not have classes or subtype inheritance.

What does this mean?.

No type hierarchy

O-O is important because it provides uniformity of interface.

Problem: subtype inheritance encourages non-uniform interfaces

Why?

- Design by type inheritance oversold.
- Early decisions hard to change, often poorly informed.
- Makes every programmer an interface designer.
- Therefore encourages overdesign early on: predict every eventuality.
- Exacerbates the problem, complicates designs.

Go inheritance vs composition

- Go does not support inheritance, it does support composition.
- The generic definition of composition is "put together" one struct field is embedded in another,
- Go gives us the option to access the embedded fields as if they were part of the outer struct

```
type Point struct {  
    X float64  
    Y float64  
}
```

Struct che si chiama Point

Non ci sono costruttori predefiniti in Go

```
func main() {  
    punto := Point{3,5}  
    p := &Point{ 3, 4 }  
    fmt.Println(p) //print {3,4}  
    fmt.Print(punto)//print{3,5}  
}
```

```
func CreateCopy(x,y float64) Point {  
    return Point{x,y}  
}
```

```
func CreateCopy2(x,y float64) *Point {  
    return &Point{x,y}  
}
```

```
func main() {  
    punto := CreateCopy(3,5)  
    p := &Point{ 3, 4 }  
    fmt.Println(p) //print {3,4}  
    fmt.Print(punto)//print{3,5}  
}
```

```
func main() {  
  
    p := CreateCopy2(3, 4)  
    fmt.Println(p) //print {3,4}  
  
}
```

new

new is a built_in function

new(t)

```
p := new(int)
fmt.Println(*p) // 0 (zero value)
*p = 42
fmt.Println(*p) // 42
```

```
type Point struct {
    X, Y int
}

p := new(Point)
fmt.Println(p.X, p.Y) // 0 0
```

p is a pointer

Method vs. function

Methods

- A method is just a function with a special receiver type between the func keyword and the method name. The receiver can either be a struct type or non-struct type.
- The syntax of a method declaration is provided below.

```
func (t Type) methodName(parameter list) { }
```

Methods on structs: Pointer Receivers vs Value Receivers

Go has no classes, but you can attach methods to any type. Yes, (almost) any type.

The methods are declared, separate from the type declaration, as functions with an explicit receiver.

The obvious struct case:

```
type Point struct { x, y float64 }  
// A method on *Point  
func (p *Point) Abs() float64 {  
    return math.Sqrt(p.x*p.x + p.y*p.y)  
}
```

Pointer Receivers

Note: explicit receiver (no automatic this), in this case of type `*Point`, used within the method.

Methods on struct values

```
type Point3 struct { x, y, z float64 }  
// A method on Point3  
func (p Point3) Abs() float64 {  
    return math.Sqrt(p.x*p.x + p.y*p.y + p.z*p.z)  
}
```

Value Receivers

- A method does not require a pointer as a receiver.
- This is a bit expensive, because the Point3 will always be passed to the method by value, but it is valid Go.

Invoking a method

Just as you expect.

```
p := &Point{ 3, 4 }  
fmt.Print(p.Abs()) // will print
```

A non-struct example:
))

```
type IntVector []int  
func (v IntVector) Sum() (s int) {  
    for _, x := range v { // blank identifier!  
        s += x  
    }  
    return  
}  
fmt.Println(IntVector{1, 2, 3}.Sum())
```

Ground rules for methods

- Methods are attached to a named type, say T, and are statically bound.
- The type T must be defined in the same package as all its methods.
 - The type of a receiver in a method can be either *T or T.
 - You can have some T methods and some *T methods.
 - T itself cannot be a pointer type, although the methods can have receiver type *T.

Pointers and values

- Go automatically indirects/dereferences values for you when invoking methods.

- For instance, even though a method has receiver type `*Point` you can invoke it on

```
type Point.  
p1 := Point{ 3, 4 }  
fmt.Print(p1.Abs()) // sugar for (&p1
```

```
).Abs()
```

- Similarly, if `r` is a variable of type `*Point3`,

```
p3 := &Point3{ 3, 4, 5 }  
fmt.Print(p3.Abs()) // sugar for (*p3).Abs()
```

value of type

Methods on anonymous fields

When an anonymous field is embedded in a struct, the methods of that type are embedded as well - in effect, it **inherits** the methods.

This mechanism offers a simple way to emulate some of the effects of subclassing and inheritance.

[illegible]

Overriding a method : embedding

Overriding works just as with fields.

- Of course you can have multiple anonymous fields with various types - a simple version of multiple inheritance.
- The conflict resolution rules keep things simple, though.

```
// Parent object
type Point struct { x, y float64 }
func (p *Point) M() float64 { ... }

//type ObjectChild
type NamedPoint struct {
    Point
    name string
}
func (p * NamedPoint) M() float64 { ... }
```

```
func main() {
    p1 := Point{}
    p1.M()
    p2 := NamedPoint()
    p2.M()
}
```

Overriding a method

Overriding works just as with fields.

- Of course you can have multiple anonymous fields with various types - a simple version of multiple inheritance.
- The conflict resolution rules keep things simple, though.

```
// Parent object
type Point struct { x, y float64 }
func (p *Point) M() float64 { ... }

//type ObjectChild
type NamedPoint struct {
    Point
    name string
}
func (p * NamedPoint) M() float64 { ... }
```

```
func main() {
    p1 := Point{}
    p1.M()
    p2 := NamedPoint()
    p2.M()
}
```

Embedding

```
type A struct{}

func (A) Say() {
    fmt.Println("A.Say")
}

type B struct {
    A
}

func (B) Say() {
    fmt.Println("B.Say")
}
```

```
func main() {

    b := B{}
    b.Say() // stampa: B.Say

}
```

Esempio

```
package main
```

```
import (  
    "fmt"  
)
```

```
type Point struct{x,y int}
```

```
func (p Point) modify() {
```

```
    p.x = p.x + 2
```

```
    p.y = p.y + 2
```

```
}
```

```
func (p *Point) modify2() {
```

```
    p.x = p.x + 2
```

```
    p.y = p.y + 2
```

```
}
```

```
func main() {
```

```
    PA := Point{0,0}
```

```
    PPA := &Point{3,4}
```

```
    fmt.Println(PA)
```

```
    fmt.Println(PPA)
```

```
    fmt.Println(PA.x, " ", PA.y)
```

```
    fmt.Println(PPA.x, " ", PPA.y)
```

```
    PA.modify()
```

```
    fmt.Println(PA.x, " ", PA.y)
```

```
    PA.modify2()
```

```
    fmt.Println(PA.x, " ", PA.y)
```

```
    PPA.modify()
```

```
    fmt.Println(PPA.x, " ", PPA.y)
```

```
    PPA.modify2()
```

```
    fmt.Println(PPA.x, " ", PPA.y)
```

```
}
```

Output

{0 0}

{3 4}

0 0

3 4

0 0

2 2

3 4

5 6

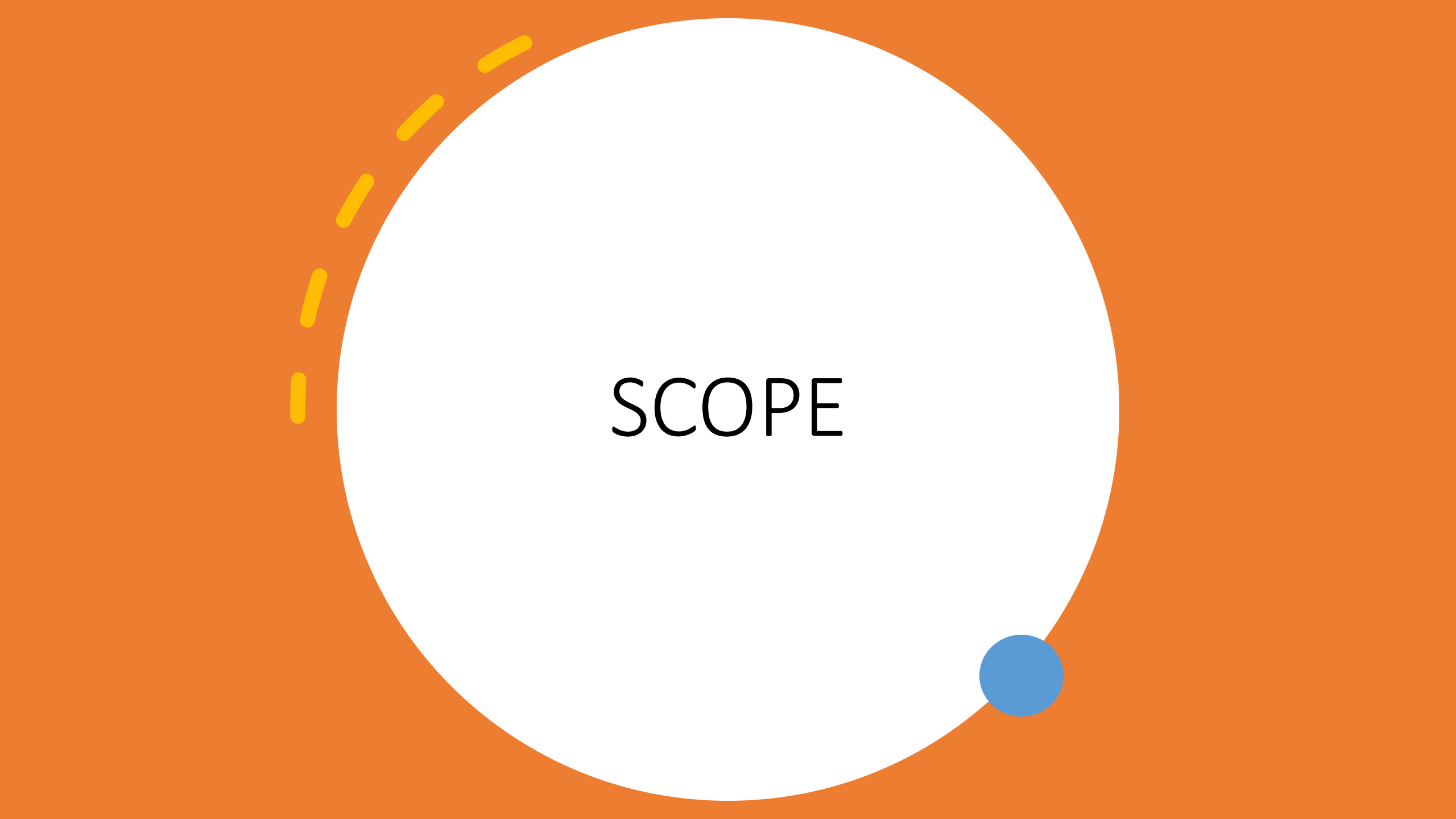
IOTA CONSTANT

```
const (  
  a = iota  
  b  
  c  
)
```

```
const (  
  daysOfWeek = [7]string{  
    iota, "Lunedì", "Martedì", "Mercoledì", "Giovedì", "Venerdì",  
    "Sabato", "Domenica",  
  }  
)
```

```
const (  
  KB = 1 << (10 * iota) // 1 << 0 = 1  
  MB           // 1 << 10  
  GB           // 1 << 20  
  TB           // 1 << 30  
)
```

```
const (  
  A = iota // 0  
  _        // skip 1  
  C        // 2  
)
```



SCOPE

Scope

Go has very simple scope hierarchy:

- universe
- package
- file (for imports only)
- function
- Block

Go's rules:

1. Go has package scope (C++ has file scope).
 2. Spelling determines exported/local (pub/priv).
 3. Structs in the same package have full access to one another's fields and methods.
 4. Local type can export its fields and methods.
 5. No true subclassing, so no notion of "protected".
- These simple rules seem to work well in practice.

Locality of naming

Nuances:

- no implicit this in methods (receiver is explicit);
- package qualifier always present for imported names (first component of) every name is always declared in current package

No surprises when importing:

- adding an exported name to my package cannot break your package!

Names do not leak across boundaries.

- In C, C++, Java the name `y` could refer to anything
- In Go, `y` (or even `Y`) is always defined within the package.
- In Go, `x.Y` is clear: find `x` locally, `Y` belongs to it.

Function and method lookup

- Method lookup by name only, not type.
- A type cannot have two methods with the same name, ever (no overload).
- Easy to identify which function/method is referred to.
- Simple implementation, simpler program, fewer surprises.
- Given a method `x.M`, there's only ever one `M` associated with `x`.



Interface

Interfaces

Interface as a concept is very close to that of Java

Java has an interface type, but the "interface value" concept of Go is novel.

Composition not inheritance

The word "interface" is a bit overloaded in Go:

- there is the concept of an interface, and there is an interface type, and then there are values of that type. First, the concept.

Definition:

- An interface is a set of methods.
- To turn it around, the methods implemented by a concrete type such as a struct form the interface of that type.

Go: interface composition

In Go an interface is just a set of methods:

```
type Hash interface {  
    Write(p []byte) (n int, err error)  
    Sum(b []byte) []byte  
    Reset()  
    Size() int  
    BlockSize() int  
}
```

No implements declaration.

All hash implementations satisfy this implicitly. (Statically checked.)

Example

We saw this simple type before:

```
type Point struct { x, y float64 }  
func (p *Point) Abs() float64 { ... }
```

The interface of type Point has the method:

```
Abs() float64
```

It's not

```
func (p *Point) Abs() float64
```

because the interface abstracts away the receiver.

We embedded Point in a new type, NamedPoint;

NamedPoint has the same interface.

Interfaces in practice: composition

Tend to be small: one or two methods are common.

Composition falls out trivially. Easy example, from packageio:

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

Reader (plus the complementary Writer) makes it easy to chain: files, buffers, networks, encryptors, compressors, GIF, JPEG, PNG, ...

Dependency structure is not a hierarchy; these also implement other interfaces.

Growth through composition is natural, does not need to be pre-declared.

Many to many

An interface may be implemented by an arbitrary number of types.

AbsInterface is implemented by any type that has a method with signature `Abs() float64`, regardless of what other methods that type may have.

A type may implement an arbitrary number of interfaces. Point implements at least these two:

```
type AbsInterface interface { Abs() float64 }
```

```
type EmptyInterface interface { }
```

And perhaps more, depending on its methods.

Every type implements EmptyInterface. That will come in handy.

Interface value

- Once a variable is declared with interface type, it may store any value that implements that interface.

```
var ai AbsInterface
pp := new(Point)
ai = pp           // OK: *Point has Abs()
ai = 7.           // compile-time err:
// float64 has no Abs()
ai = MyFloat(-7.) // OK: MyFloat has Abs()
ai = &Point{ 3, 4 }
fmt.Printf(ai.Abs()) // method call Prints 5.
```

- Note: ai is not a pointer! It is an interface value

```
package main
```

```
import "fmt"
```

```
// Definizione di un'interfaccia chiamata Shape
```

```
type Shape interface {
```

```
    Area() float64
```

```
    Perimeter() float64
```

```
}
```

```
// Definizione di un tipo Rectangle che implementa l'interfaccia Shape
type Rectangle struct {
    Width float64
    Height float64
}

// Implementazione dei metodi dell'interfaccia per il tipo Rectangle
func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2*r.Width + 2*r.Height
}

// Funzione che accetta un parametro di tipo Shape
func PrintShapeInfo(s Shape) {
    fmt.Printf("Area: %f\n", s.Area())
    fmt.Printf("Perimeter: %f\n", s.Perimeter())
}
```

```
func main() {  
    // Creazione di un oggetto Rectangle  
    rectangle := Rectangle{Width: 5, Height: 10}  
  
    // Chiamata alla funzione PrintShapeInfo passando l'oggetto  
    Rectangle  
    PrintShapeInfo(rectangle)  
}
```

Sort interface

- sort interface defines three methods
- any type that implements those three methods can sort
- algorithms are inside the sort package, invisible outside

```
package sort
```

```
type Interface interface {  
    Len() int  
    Less(i, j int) bool  
    Swap(i, j int)  
}
```

Sort interface

(adapted from Go Tour)

```
type Person struct {  
    Name string  
    Age  int  
}  
  
func (p Person) String() string {  
    return fmt.Sprintf("%s: %d", p.Name, p.Age)  
}
```

Sort interface (adapted from Go Tour)

```
type ByAge []Person

func (a ByAge) Len() int           { return len(a) }

func (a ByAge) Swap(i, j int)      {
    a[i], a[j] = a[j], a[i] }

func (a ByAge) Less(i, j int) bool {
    return a[i].Age < a[j].Age }

func main() {
    people := []Person{"Bob", 31}, {"Sue", 42}, {"Ed", 17}, {"Jen", 26},
    fmt.Println(people)
    sort.Sort(ByAge(people))
    fmt.Println(people)
}
```

Polymorphism using interface

- **A variable of type interface can hold any value which implements the interface. This property of interfaces is used to achieve polymorphism in Go.**

Polimorfism

```
type Speaker interface {  
    Say()  
}  
  
type A struct{}  
func (A) Say() { fmt.Println("A") }  
  
type B struct{}  
func (B) Say() { fmt.Println("B") }
```

```
var s Speaker  
  
s = A{}  
s.Say() // A  
  
s = B{}  
s.Say() // B
```

Go vs OO

Structs: Structs are the basic building blocks of objects in Golang. They define the data and behavior of an object by encapsulating fields and methods.

Methods: Methods are functions associated with structs that operate on the data within those structs. They encapsulate the behavior of objects.

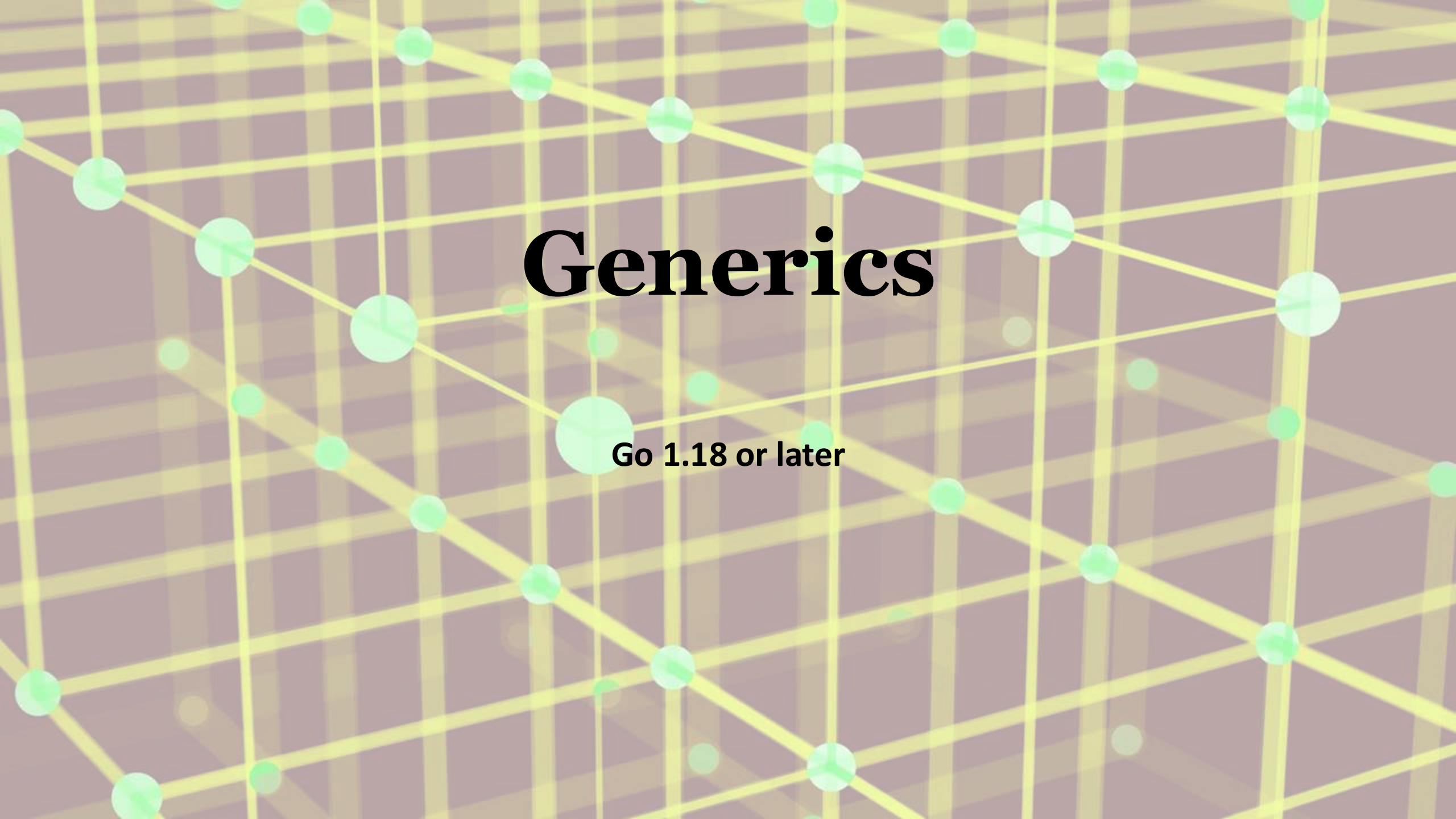
Interfaces: Interfaces define sets of methods that structs can implement. They promote polymorphism and enable loose coupling between objects.

Embedded Types: Embedded types allow structs to contain other structs within them. This facilitates composition, which is the preferred approach to code reuse in Golang.

Pointers: Pointers provide a way to reference objects in memory. They are essential for pointer receivers, which are methods that receive a pointer to the struct they operate on.

While Golang supports these OOP features, it de-emphasizes some traditional OOP concepts:

- **Inheritance:** Golang discourages the use of inheritance and instead favors composition. Composition allows objects to be composed from other objects, creating a more flexible and maintainable code structure.
- **Abstract Classes:** Golang does not have abstract classes, but it achieves similar functionality through interfaces and anonymous structs. This approach promotes code reuse without the complexity of abstract classes.
- **Access Modifiers:** Golang does not have explicit access modifiers like public, private, or protected. Instead, it relies on conventions and naming practices to control access to fields and methods.



Generics

Go 1.18 or later

Generic

Generics are a way of writing code that is independent of the specific types being used. Functions and types may now be written to use any of a set of types.

Generics add three new big things to the language:

- Type parameters for function and types.
- Defining interface types as sets of types, including types that don't have methods.
- Type inference, which permits omitting type arguments in many cases when calling a function.

Type parameters

`Print[int]([int]{1, 2, 3})`

```
func Print[T any](s []T) {  
    for _, v := range s {  
        fmt.Println(v) }  
}
```

This type parameter list appears before the regular parameters.

To distinguish the type parameter list from the regular parameter list, the type parameter list uses square brackets rather than parentheses.

Just as regular parameters have types, type parameters have meta-types, also known as constraints.

Use of Type parameters

```
package main

import "fmt"

// Funzione generica con un parametro di tipo T
func PrintSlice[T any](s []T) {
    for _, v := range s {
        fmt.Println(v)
    }
}

func main() {
    nums := []int{1, 2, 3}
    strs := []string{"a", "b", "c"}

    PrintSlice(nums) // Stampa interi
    PrintSlice(strs) // Stampa stringhe
}
```

Use of Type parameters

```
package main

import "fmt"

// Funzione generica con un parametro di tipo T
func PrintSlice[T any](s []T) {
    for _, v := range s {
        fmt.Println(v + 1)
    }
}

func main() {
    nums := []int{1, 2, 3}
    strs := []string{"a", "b", "c"}

    PrintSlice(nums) // Stampa interi
    PrintSlice(strs) // Stampa stringhe
}
```

invalid operation: v + 1 (mismatched types T and untyped int)

Multiple type parameters

```
func Print2[T1, T2 any](s1 []T1, s2 []T2) { ... }
```

```
package main
```

```
import "fmt"
```

```
func Swap[T, U any](a T, b U) (U, T) {  
    return b, a  
}
```

```
func main() {  
    a, b := Swap(42, "Generici")  
    fmt.Println(a, b) // Generici 42  
}
```

Generic interface

```
type MyInterface[T any] interface {  
    DoSomething(x T) }
```

```
type MyInterface[T any] interface {  
    DoSomething(x T)  
}
```

```
type MyType[T any] struct {  
}
```

```
func (t *MyType[T]) DoSomething(x T) {  
    // ...  
}
```

Operations permitted for any type

- declare variables of those types
- assign other values of the same type to those variables
- pass those variables to functions or return them from functions
- take the address of those variables
- convert or assign values of those types to the type `interface{}`
- convert a value of type `T` to type `T` (permitted but useless)
- use a type assertion to convert an interface value to the type
- use the type as a case in a type switch
- define and use composite types that use those types, such as a slice of that type
- pass the type to some predeclared functions such as `new`

Constrainst

```
package main
```

```
import "fmt"
```

```
type Number interface {  
    int | float64 | int32 | int64  
}
```

```
func PrintSlice[T Number](s []T) {  
    for _, v := range s {  
        fmt.Println(v + 1)  
    }  
}
```

```
func main() {  
    nums := []int{1, 2, 3}  
    PrintSlice(nums) // Stampa numeri incrementati di 1  
}
```

Constraints

```
// Stringer is a type constraint that requires a String method.  
// The String method should return a string representation of the value.  
type Stringer interface {  
    String() string  
}
```

```
// Plusser is a type constraint that requires a Plus method.  
// The Plus method is expected to add the argument to an internal  
// string and return the result.  
type Plusser interface {  
    Plus(string) string  
}
```

```
func ConcatTo[S Stringer, P Plusser](s []S, p []P) []string {  
    r := make([]string, len(s))  
    for i, v := range s {  
        r[i] = p[i].Plus(v.String())  
    }  
    return r  
}
```


A single constraint can be used for multiple type parameters, just as a single type can be used for multiple non-type function parameters. The constraint applies to each type parameter separately.

```
// Stringify2 converts two slices of different types to strings,  
// and returns the concatenation of all the strings.
```

```
func Stringify2[T1, T2 Stringer](s1 []T1, s2 []T2) string {  
    r := ""  
    for _, v1 := range s1 {  
        r += v1.String()  
    }  
    for _, v2 := range s2 {  
        r += v2.String()  
    }  
    return r  
}
```

Multiple parameter

- A single constraint can be used for multiple type parameters, just as a single type can be used for multiple non-type function parameters.
- The constraint applies to each type parameter separately.

Type of constraints

- In Go, **type constraints** in generics specify **which types are allowed** as type parameters for a generic function, method, or type. They act as rules that the type parameter must satisfy.

Basic Constraint with any

```
func PrintSlice[T any](s []T) {  
    for _, v := range s {  
        fmt.Println(v)  
    }  
}
```

Constrain with interface

```
type Stringer interface {  
    String() string  
}  
  
func PrintStringer[T Stringer](s T) {  
    fmt.Println(s.String())  
}
```

Interfaces define required methods

```
type Stringer interface {  
    String() string  
}
```

Interfaces can define type sets (lists of allowed types)

```
type Integer interface {  
    int | int8 | int16 | int32 | int64  
}
```

The ~ prefix

```
type MyInt int
```

```
type Integer interface {  
    ~int  
}
```

Now `MyInt` satisfies `Integer`.

Constraints without methods act as simple type sets

```
type Number interface {  
    ~int | ~float64  
}
```

- In Go, **type constraints can include methods**. This allows you to require that a type parameter not only belongs to a set of types but also **implements specific methods**. This is very similar to “interface constraints” in Go generics.

```
// Define a constraint interface with a method
type Stringer interface {
    String() string
}
```

```
// Generic function that requires the type to implement
Stringer
func PrintString[T Stringer](v T) {
    fmt.Println(v.String())
}
```

```
// Example type that satisfies Stringer
type Person struct {
    Name string
}
```

```
func (p Person) String() string {
    return p.Name
}
```

```
func main() {
    p := Person{Name: "Alice"}
    PrintString(p) // Output: Alice
}
```

Custom Constraints

```
type MyConstraint interface {  
    int | string  
}
```

Comparable constraint

```
func Contains[T comparable](slice []T, item T) bool {  
    for _, v := range slice {  
        if v == item {  
            return true  
        }  
    }  
    return false  
}
```

Constraint with includes interface

```
import "fmt"

func PrintStringer[T fmt.Stringer](s T) {
    fmt.Println(s.String())
}
```

Generic type

```
package main  
import "fmt"
```

```
func MapKeys[K comparable, V any](m map[K]V) []K {  
    r := make([]K, 0, len(m))  
    for k := range m {  
        r = append(r, k)  
    }  
    return r  
}
```

```
type List[T any] struct {  
    head, tail *element[T]  
}
```

```
type element[T any] struct {  
    next *element[T]  
    val T  
}
```

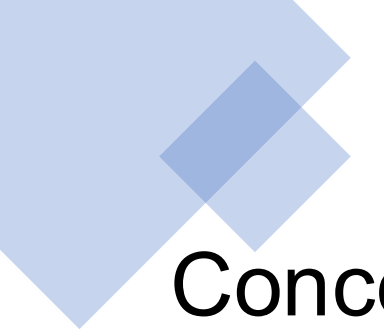
```
func (lst *List[T]) Push(v T) {  
    if lst.tail == nil {  
        lst.head = &element[T]{val: v}  
        lst.tail = lst.head  
    } else {  
        lst.tail.next = &element[T]{val: v}  
        lst.tail = lst.tail.next  
    }  
}
```

```
func (lst *List[T]) GetAll() []T {  
    var elems []T  
    for e := lst.head; e != nil; e = e.next {  
        elems = append(elems, e.val)  
    }  
    return elems  
}
```

```
func main() {  
    var m = map[int]string{1: "2", 2: "4", 4: "8"}  
  
    fmt.Println("keys:", MapKeys(m))  
    _ = MapKeys[int, string](m)  
    lst := List[int]{}  
    lst.Push(10)  
    lst.Push(13)  
    lst.Push(23)  
    fmt.Println("list:", lst.GetAll())  
}
```


Built-in constraints (from constraints package)

- `constraints.Ordered` → types supporting `<`, `<=`, `>`, `>=` (`int`, `float`, `string`)
- `constraints.Integer` → all integer types
- `constraints.Float` → `float32`, `float64`



Concept

Description

Type constraint

An interface defining allowed types

Type set

List of types or underlying types
allowed

$\sim T$

Any type with underlying type T

$\backslash$

$\backslash$

Built-in constraints

`Ordered`, `Integer`, `Float`





First class function and Funcional Programming

What are first class functions?

A language which supports first class functions allows functions to be assigned to variables, passed as arguments to other functions and returned from other functions. Go has support for first class functions.

- Anonymous functions**
- User defined function types**
- Higher-order functions**
- Closures**

Anonymous function

```
package main
```

```
import (  
    "fmt"  
)
```

```
func main() {  
    a := func() {  
        fmt.Println("hello world first class function")  
    }  
    a()  
    fmt.Printf("%T", a)  
}
```

These kind of functions are called anonymous functions since they do not have a name.

The only way to call this function is using the variable a. We have done this in the next line. a() calls the function

Anonymous function

```
package main
```

```
import (  
    "fmt"  
)
```

```
func main() {  
    func(n string) {  
        fmt.Println("Welcome", n)  
    }("Gophers")  
}
```

immediately after the function definition, we call the function using ()

User defined function types

it is possible to define
our own function
types.

**type add func(a int, b
int) int**

**Now we can define
variables of type add.**

```
package main

import (
    "fmt"
)

type add func(a int, b int) int

func main() {
    var a add = func(a int, b int) int {
        return a + b
    }
    s := a(5, 6)
    fmt.Println("Sum", s)
}
```

Sum 11

```

package main
import (
    "fmt"
)
func simple(a func(a, b int) int) {
    fmt.Println(a(60, 7))
}
func main() {
    f := func(a, b int) int {
        return a + b
    }
    g := func(a, b int) int {
        return a * b
    }
    simple(f)
    simple(g)
}

```

► The definition of Higher-order function from wiki is a function which does at least one of the following

- takes one or more functions as arguments
- returns a function as its result

67
420

67

```
package main
```

```
import (  
    "fmt"  
)
```

```
func simple() func(a, b int) int {  
    f := func(a, b int) int {  
        return a + b  
    }  
    return f  
}
```

```
func main() {  
    s := simple()  
    fmt.Println(s(60, 7))  
}
```

Clousure

Closures are anonymous functions which access the variables defined outside the body of the function.

```
package main
```

```
import (  
    "fmt"  
)
```

```
func main() {  
    a := 5  
    func() {  
        fmt.Println("a =", a)  
    }()  
}
```

```
a = 5
```



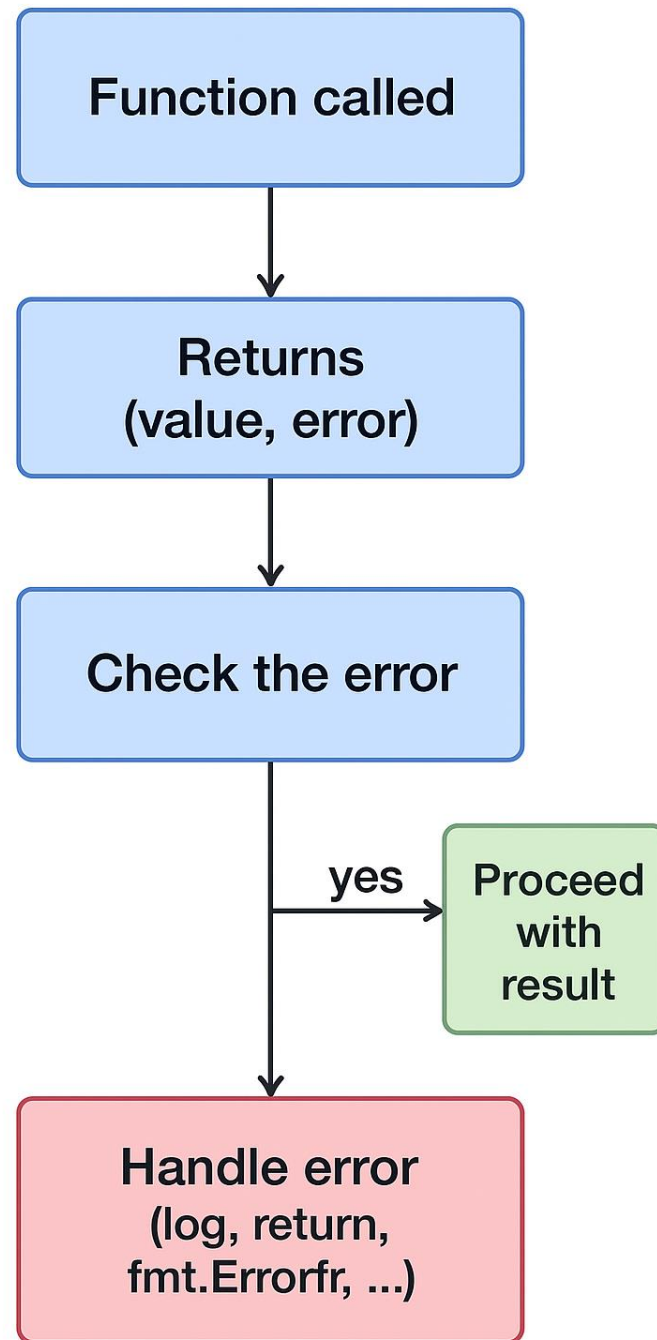
Exception Handling

Error

- Errors in Go are plain old values.
- Errors are represented using the built-in **error** type
- Just like any other built-in type such as int, float64, ... error values can be stored in variables, passed as parameters to functions, returned from functions, and so on.

```
type error interface {  
    Error() string  
}
```

It contains a single method with signature `Error() string`. Any type which implements this interface can be used as an error. This method provides the description of the error.



ERROR package

- The most commonly-used error implementation is the errors package's unexported errorString type.
- Package errors implements functions to manipulate errors.
- The **New** function creates errors whose only content is a text message.

func New(text string) error

- New returns an error that formats as the given text. Each call to New returns a distinct error value even if the text is identical.

```
import (  
    "errors"  
    "fmt"  
)
```

```
func main() {  
    err := errors.New("errore")  
    if err != nil {  
        fmt.Print(err)  
    }  
}
```

ERROR package

- `func Is(err, target error) bool`

Is reports whether any error in err's chain matches target.

The chain consists of err itself followed by the sequence of errors obtained by repeatedly calling Unwrap.

An error is considered to match a target if it is equal to that target or if it implements a method `Is(error) bool` such that `Is(target)` returns true.

```
func main() {  
    if _, err := os.Open("non-existing"); err != nil {  
        if errors.Is(err, fs.ErrNotExist) {  
            fmt.Println("file does not exist")  
        } else {  
            fmt.Println(err)  
        }  
    }  
}
```

ERROR package

```
func As(err error, target interface{}) bool
```

As finds the first error in err's chain that matches target, and if so, sets target to that error value and returns true. Otherwise, it returns false.

The chain consists of err itself followed by the sequence of errors obtained by repeatedly calling Unwrap.

```
func main() {  
    if _, err := os.Open("non-existing"); err != nil {  
        var pathError *fs.PathError  
        if errors.As(err, &pathError) {  
            fmt.Println("Failed at path:", pathError.Path)  
        } else {  
            fmt.Println(err)  
        }  
    }  
}
```


ERROR package

```
func Unwrap(err error) error
```

Unwrap returns the result of calling the Unwrap method on err, if err's type contains an Unwrap method returning error. Otherwise, Unwrap returns nil.

If e.Unwrap() returns a non-nil error w, then we say that e wraps w.

Unwrap unpacks wrapped errors. If its argument's type has an Unwrap method, it calls the method once. Otherwise, it returns nil.

A simple way to create wrapped errors is to call `fmt.Errorf` and apply the `%w` verb to the error argument:

```
errors.Unwrap(fmt.Errorf("... %w ...", ..., err, ...))
```

Example 1

```
type PathError struct {
    Op  string
    Path string
    Err error
}

func (e *PathError) Error() string {
    return e.Op + " " + e.Path + ": " + e.Err.Error()
}

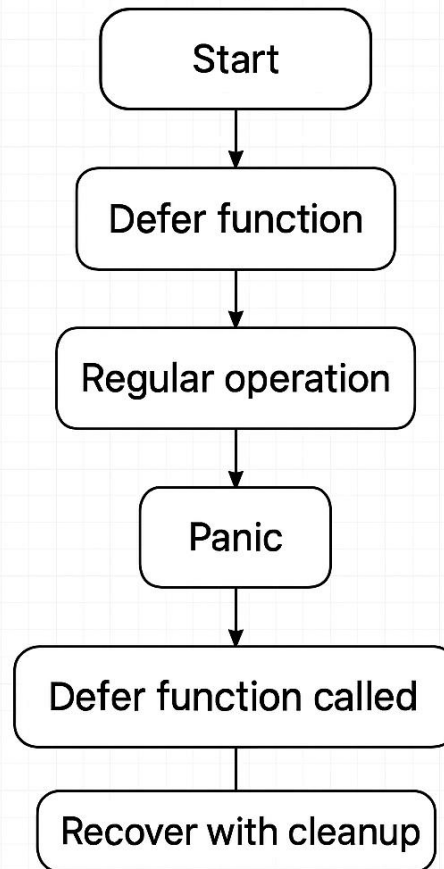
func main() {
    f, err := os.Open("test.txt")
    if err != nil {
        if pErr, ok := err.(*os.PathError); ok {
            fmt.Println("Failed to open file at path", pErr.Path)
            return
        }
        fmt.Println("Generic error", err)
        return
    }
    fmt.Println(f.Name(), "opened successfully")
}
```

Example

```
type DNSError struct {  
    ...  
}  
  
func (e *DNSError) Error() string {  
    ...  
}  
func (e *DNSError) Timeout() bool {  
    ...  
}  
func (e *DNSError) Temporary() bool {  
    ...  
}
```

```
func main() {  
    addr, err := net.LookupHost("golangbot123.com")  
    if err != nil {  
        if dnsErr, ok := err.(*net.DNSError); ok {  
            if dnsErr.Timeout() {  
                fmt.Println("operation timed out")  
                return  
            }  
            if dnsErr.Temporary() {  
                fmt.Println("temporary error")  
                return  
            }  
            fmt.Println("Generic DNS error", err)  
            return  
        }  
        fmt.Println("Generic error", err)  
        return  
    }  
    fmt.Println(addr)  
}
```

PANIC E RECOVER



```
package main
```

```
import "fmt"
```

```
func safeDivision(a, b int) {  
    defer func() {  
        if r := recover(); r != nil {  
            fmt.Println("Errore catturato:", r)  
        }  
    }()  
    fmt.Println("Risultato:", a/b)  
}
```

```
func main() {  
    safeDivision(10, 0)  
    fmt.Println("Programma continua...")  
}
```

```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("start")  
    defer func() {  
        if r := recover(); r != nil {  
            fmt.Println("recuperato panic:", r)  
        }  
    }()  
}
```

```
    mightPanic()  
    fmt.Println("end") // viene eseguito perché abbiamo recuperato il panic  
}
```

```
func mightPanic() {  
    panic("qualcosa è andato storto")  
}
```

CONVERT PANIC IN ERROR

```
func safeDo(f func() error) (err error) {
    defer func() {
        if r := recover(); r != nil {
            // r è di tipo interface{}, può essere string, error, ecc.
            if e, ok := r.(error); ok {
                err = fmt.Errorf("panic recuperato: %w", e)
            } else {
                err = fmt.Errorf("panic recuperato: %v", r)
            }
        }
    }()
    return f()
}

err := safeDo(func() error {
    // codice che può panickare
    panic("boom")
})
if err != nil {
    fmt.Println("errore:", err)
}
```